

Trees & Tree Traversals

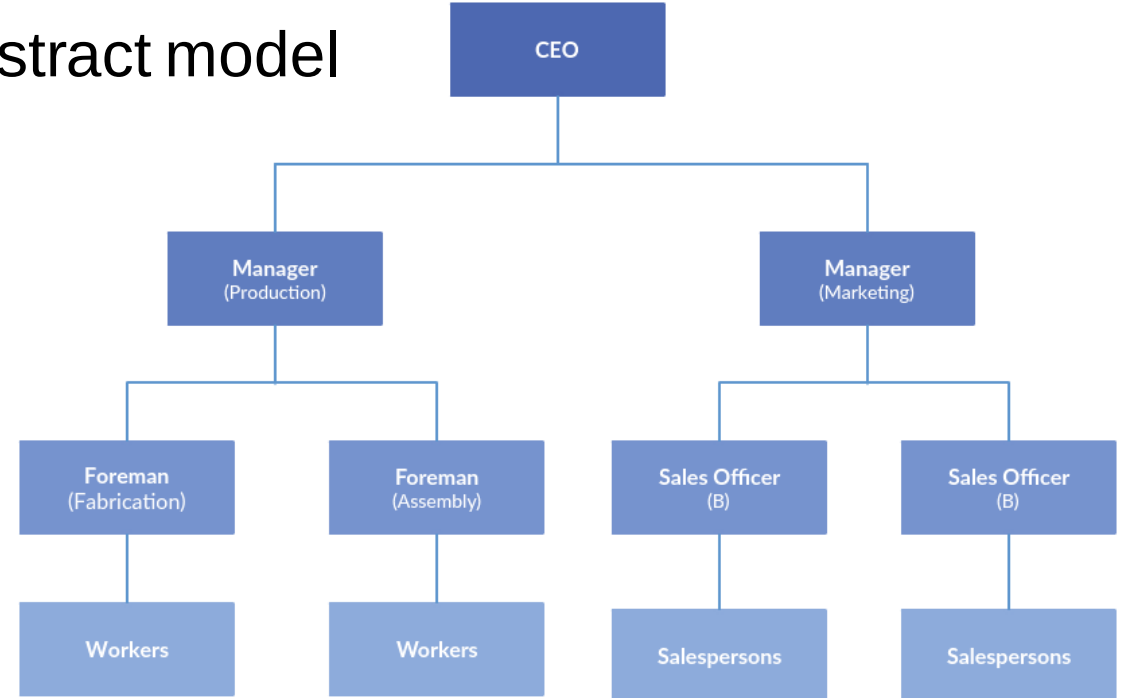
CSE 2215 - Lecture 10 – Summer 2023

Instructor : Fahmid Al Rifat, Lecturer, Dept. of CSE , UIU

fahmid@cse.uiu.ac.bd

What is a Tree?

- A connected **acyclic** graph is a tree
- In computer science, a tree is an abstract model of a hierarchical structure
- A tree consists of nodes with a parent-child relationship
- Applications:
 - Organizational charts
 - File systems
 - Programming environments

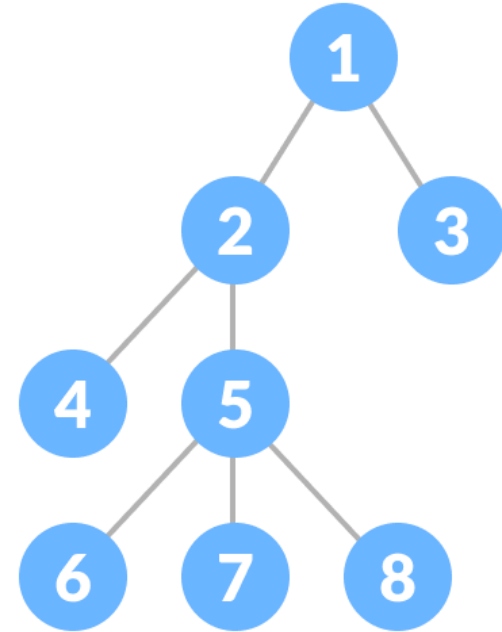


What is a Tree?

- ***A connected acyclic graph is a tree.***

- A tree T is a set of nodes in a parent-child relationship with the following properties:

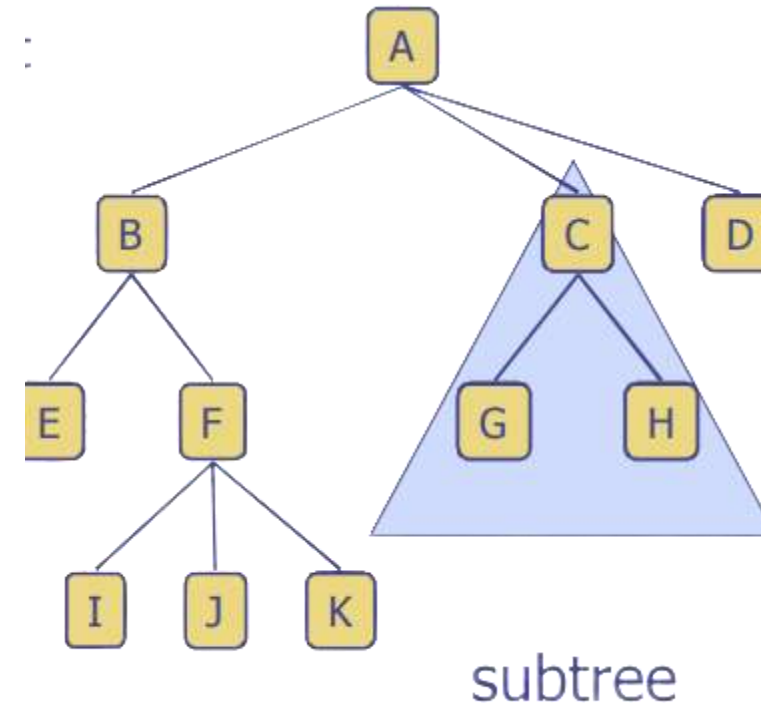
- T has a special node r , called the **root** of T , with no parent node
- Each node v of T , different from r , has a unique parent node u



Tree Terminology

- **Root**: node without parent (A)
- **Internal node**: node with at least one child (A, B, C, F)
- **External node (Leaf)**: node without children (E, I, J, K, G, H, D)
- **Ancestors of a node**: parent, grandparent, grand-grandparent, etc.
- **Descendants of a node**: child, grandchild, grand-grandchild, etc.
- **Subtree**: tree consisting of a node and its descendants

- ◆ **Height** of a tree: maximum depth of any node (3)
- ◆ **Siblings**: two nodes that are children of the same parent



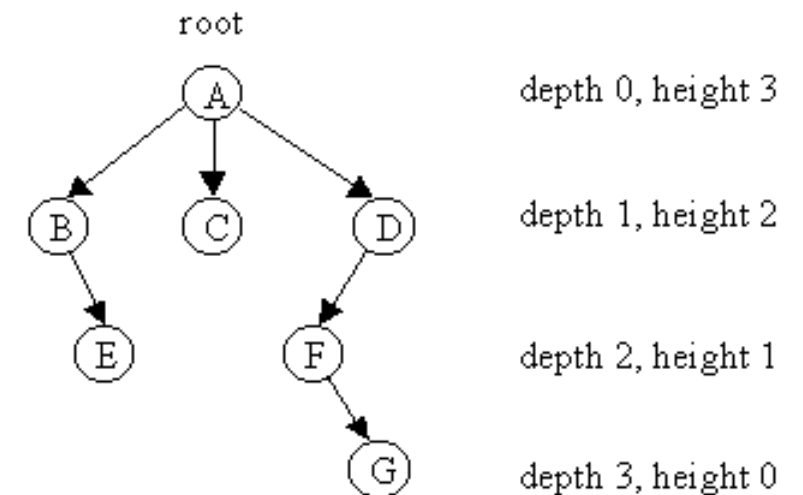
Depth

- The **depth** of a node v can be recursively defined as follows
 - If v is the root, then the depth of v is 0.
 - Otherwise, the depth of v is one plus the depth of the parent of v

Algorithm

```
depth( $T, v$ )  
if  $T.isRoot(v)$  then  
    return 0  
else  
    return 1 +  $depth(T, T.parent(v))$ 
```

- Running time: $O(1 + d_v)$, d_v is depth of v in T
- In worst case $O(n)$, n is the number of nodes in T



A tree of height 3

Height

- The *height* of a node v can be recursively defined as follows
 - If v is a leaf node, then the height of v is 0.
 - Otherwise, the height of v is one plus the maximum height of a child of v
- The height of a tree T is the height of the root of T
- The height of a tree T is equal to the maximum depth of a leaf node of T

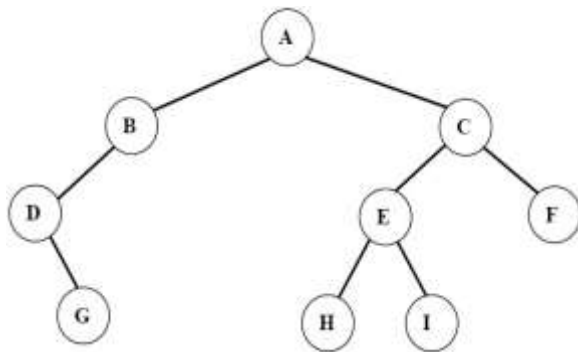
Height

```
Algorithm height( $T, v$ )  
  if  $T.isLeaf(v)$  then  
    return 0  
  else  
     $h = 0$   
    for each  $w \in T.children(v)$  do  
       $h = \max(h, \text{height}(T, w))$   
  return  $1 + h$ 
```

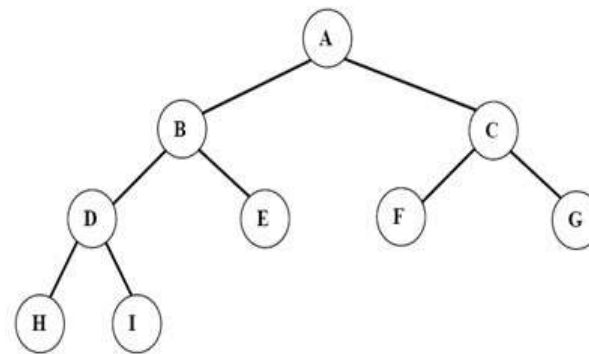
- Running time: $O(\sum_{v \in T} (1 + C_v))$, where C_v is the number of children of v in T
- Since $\sum_{v \in T} C_v = n - 1$, we have $O(\sum_{v \in T} (1 + C_v)) = O(n)$.
- The height of tree T is obtained by calling $\text{height}(T, r)$.

Ordered Trees

- A tree is ordered if there is a **linear ordering defined** for each child of each node.
- A **binary tree** is an ordered tree in which every node has **at most two children**.
- If each node of a tree has either **zero or two children**, the tree is called a **proper binary tree**.



Binary Tree



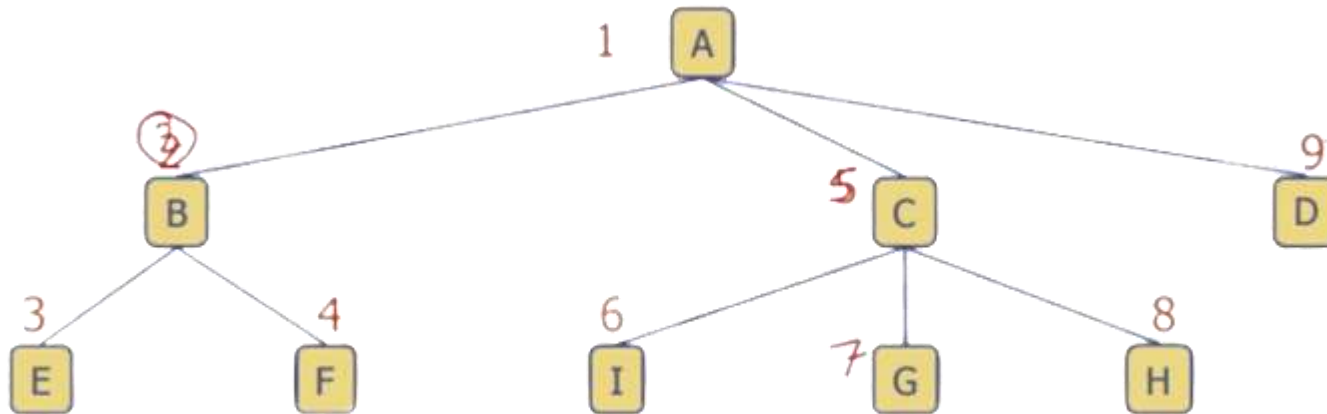
Proper Binary Tree

Traversal of Trees

- A traversal of a tree T is a systematic way of visiting all the nodes of T
- Traversing a tree involves visiting the root and traversing its subtrees
- There are the following traversal methods:
 - Preorder Traversal
 - Postorder Traversal
 - Inorder Traversal (of a binary tree)

Preorder Traversal (General Tree)

- In a preorder traversal, a node is visited before its descendants
- If a tree is ordered, then the subtrees are traversed according to the order of the children

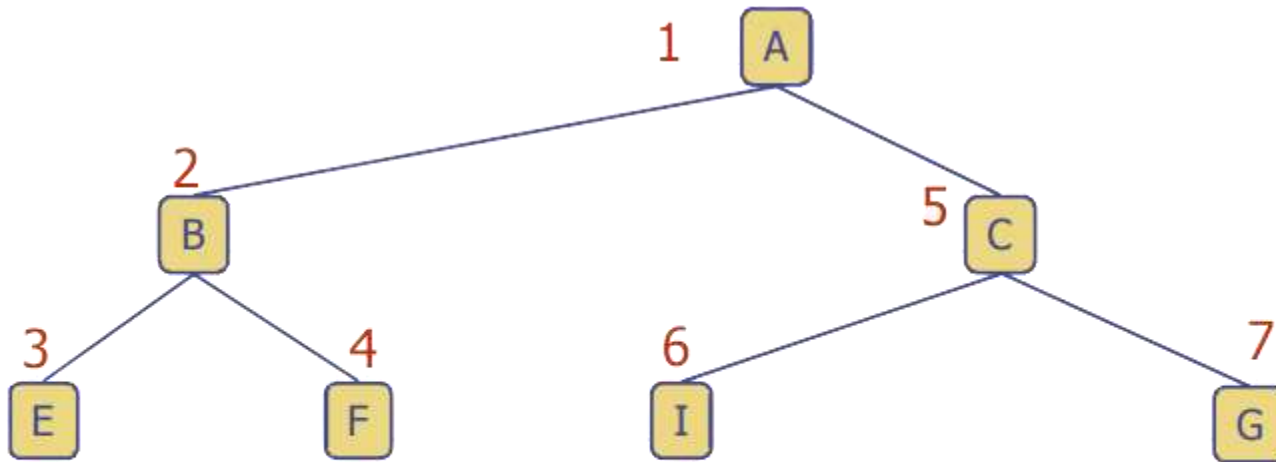


Preorder: A B E F C I G H D

Preorder Traversal (Binary Tree)

Traversing a binary tree in Preorder

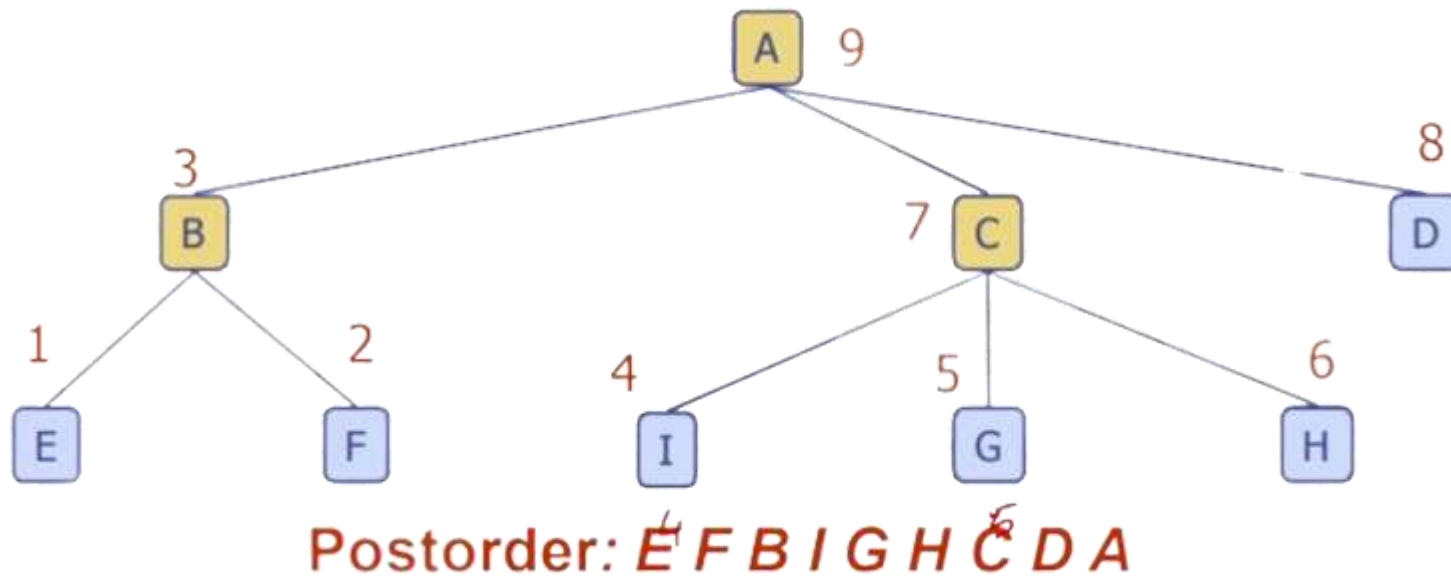
1. Visit the root.
2. Traverse the left subtree in Preorder.
3. Traverse the right subtree in Preorder.



Preorder: A B E F C I G

Postorder Traversal (General Tree)

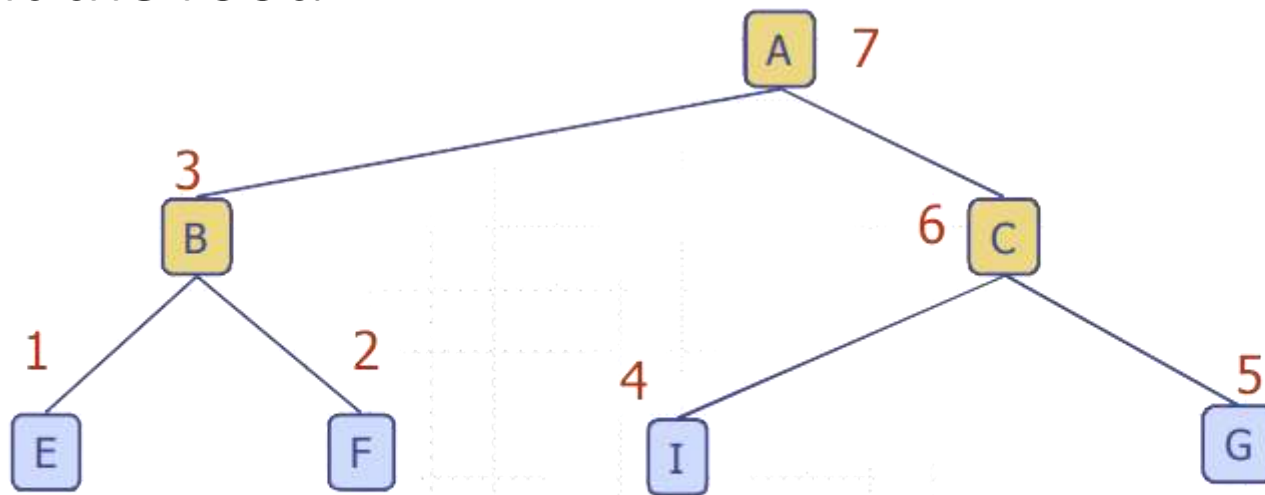
- In a postorder traversal, a node is visited after its descendants



Postorder Traversal (Binary Tree)

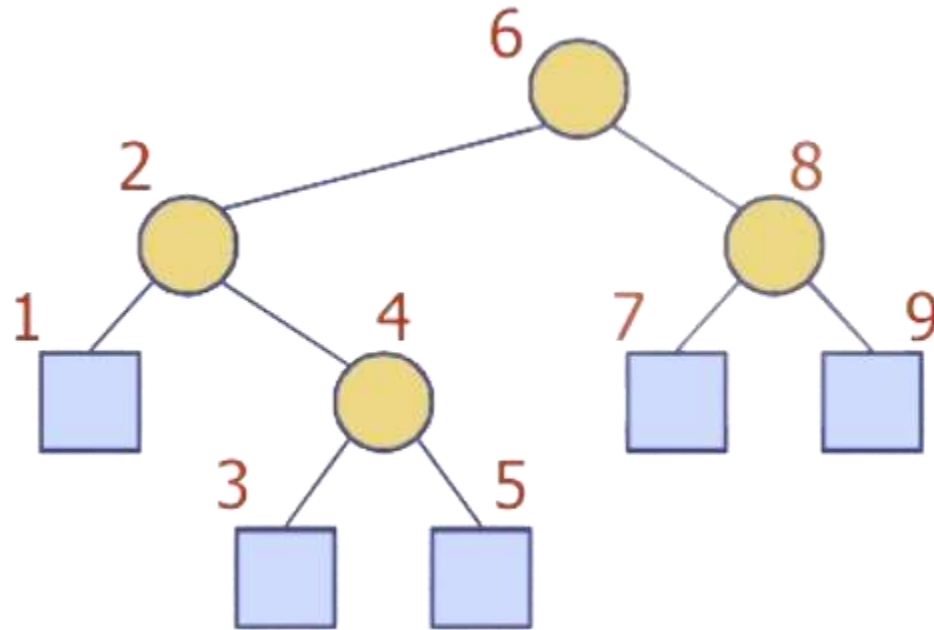
Traversing a binary tree in Postorder

1. Traverse the left subtree in Preorder.
2. Traverse the right subtree in Preorder.
3. Visit the root.



Inorder Traversal

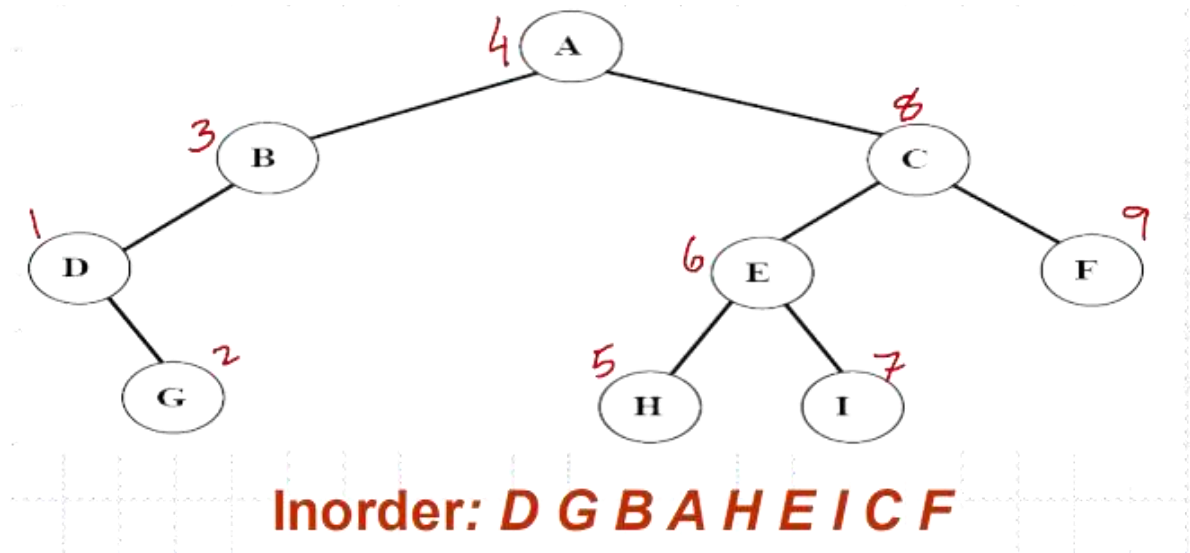
- In a postorder traversal, a node is visited after its descendants



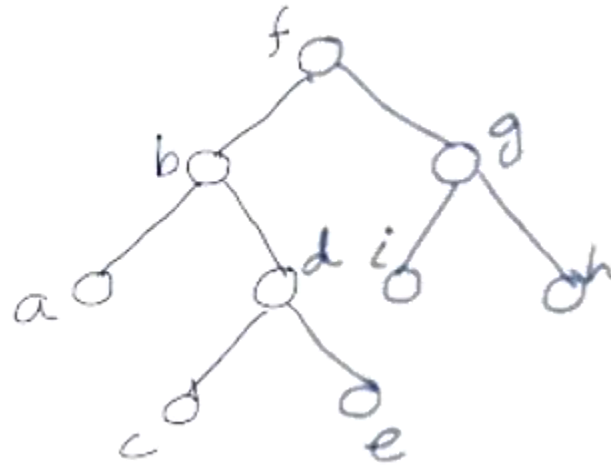
Inorder Traversal

Traversing a binary tree in inorder

1. Traverse the left subtree in inorder.
2. Visit the root.
3. Traverse the right subtree in inorder.



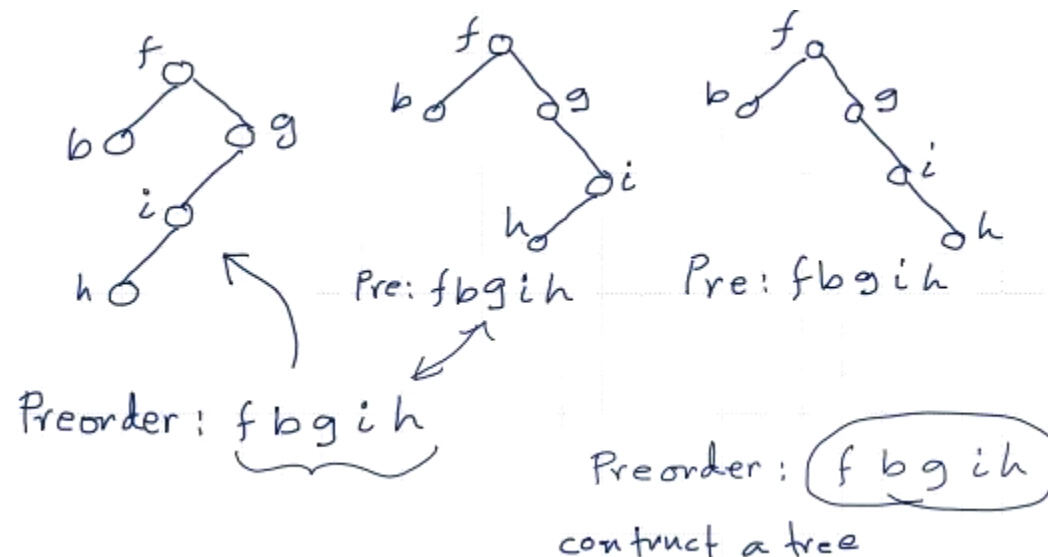
Preorder, Inorder, Postorder Traversals: Example



Preorder (root, L, R): f b a d c e g i h
Inorder (L, root, R): a b c d e f i g h
Postorder (L, R, root): a c e d b i h g f

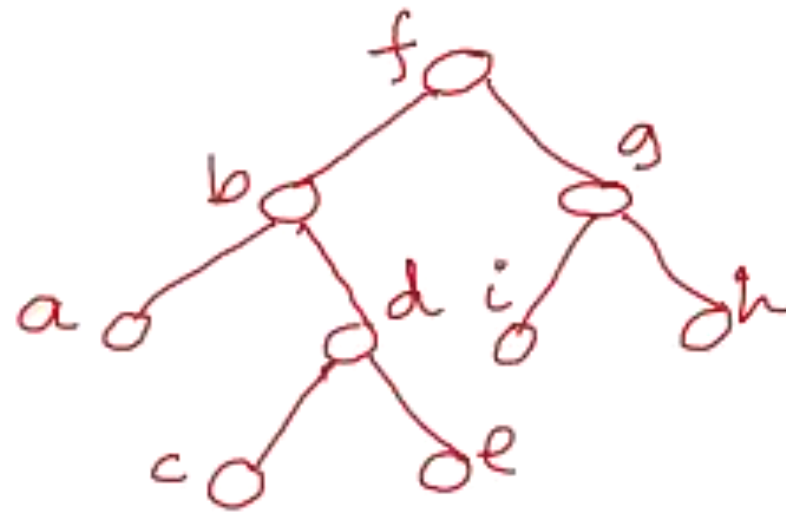
Construct a Binary Tree only from Preorder, Inorder or Postorder

- Preoder: $f \rightarrow b \rightarrow g \rightarrow i \rightarrow h$
- No Unique Tree if only one sequence is specified.
- Need at least two traversal sequences to construct a unique tree.



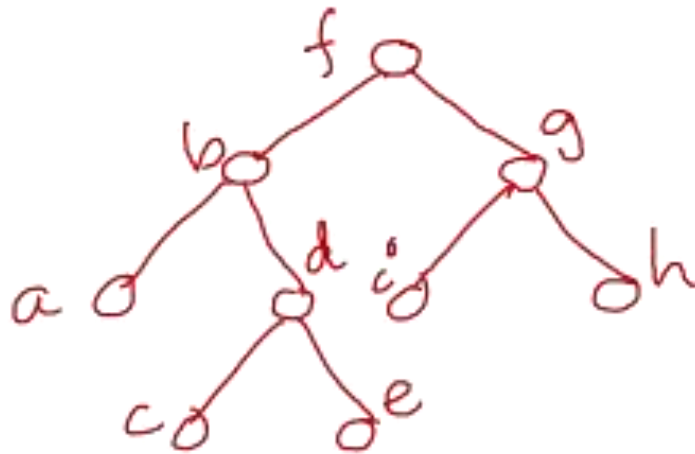
Construct a Binary Tree from *Preorder & Inorder*

- Preoder: $f \rightarrow b \rightarrow a \rightarrow d \rightarrow c \rightarrow e \rightarrow g \rightarrow i \rightarrow h$
- Inorder: $a \rightarrow b \rightarrow c \rightarrow d \rightarrow e \rightarrow f \rightarrow i \rightarrow g \rightarrow h$



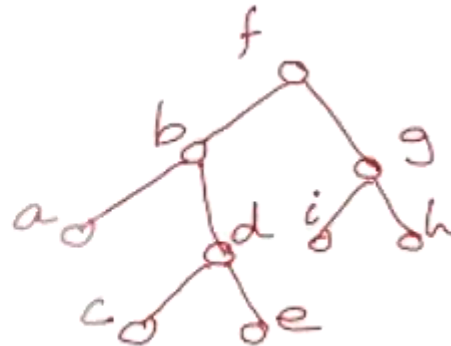
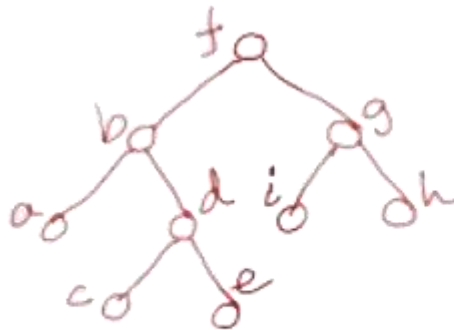
Construct a Binary Tree from *Postorder* & *Inorder*

- Postoder: $a \rightarrow c \rightarrow e \rightarrow d \rightarrow b \rightarrow i \rightarrow h \rightarrow g \rightarrow f$
- Inorder: $a \rightarrow b \rightarrow c \rightarrow d \rightarrow e \rightarrow f \rightarrow i \rightarrow g \rightarrow h$



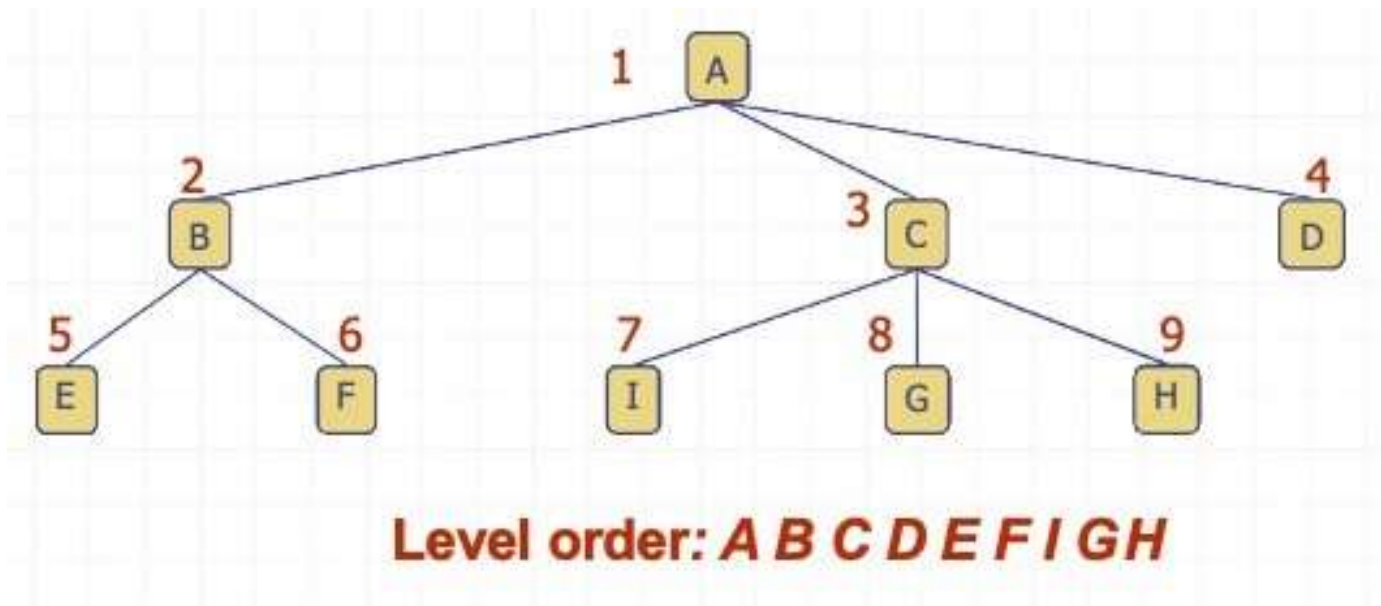
Construct a Binary Tree from *Preorder & Postorder*

- Preoder: $f \rightarrow b \rightarrow a \rightarrow d \rightarrow c \rightarrow e \rightarrow g \rightarrow i \rightarrow h$
- Postoder: $a \rightarrow c \rightarrow e \rightarrow d \rightarrow b \rightarrow i \rightarrow h \rightarrow g \rightarrow f$
- No unique Tree!!
- Need at least one Inorder traversal sequence to construct Unique tree.



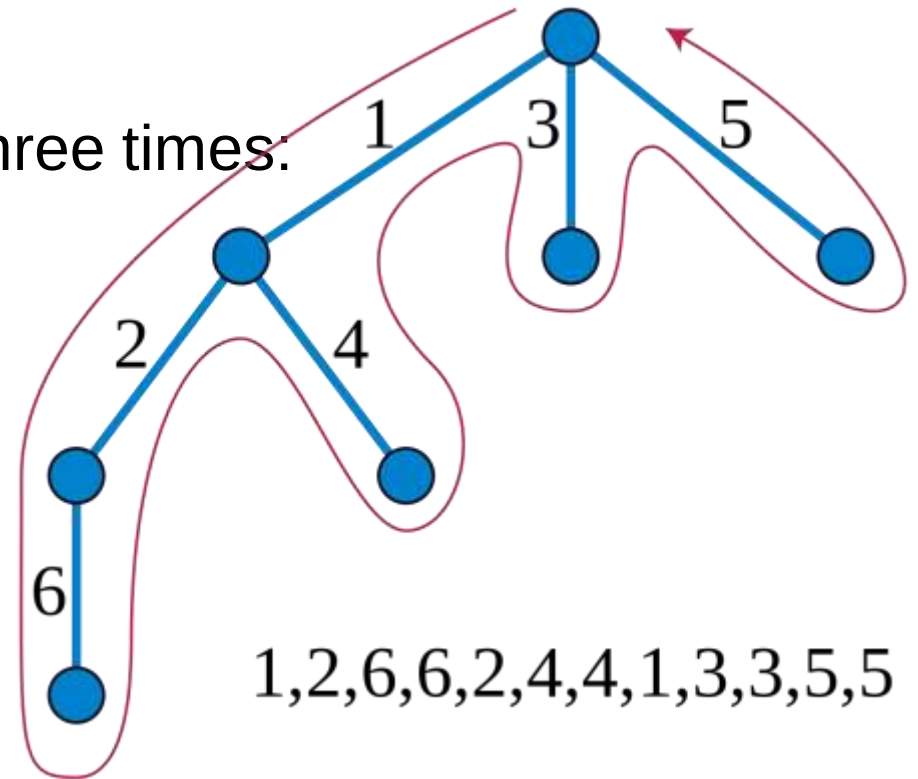
Level Order Traversal

- In a level order traversal, every node on a level is visited before going to a lower level



Euler Tour Traversal

- Generic traversal of a binary tree
- Includes a special cases the preorder, postorder and inorder traversals
- Walk around the tree and visit each node three times:
 - on the left (preorder)
 - from below (inorder)
 - on the right (postorder)



Euler Tour Traversal

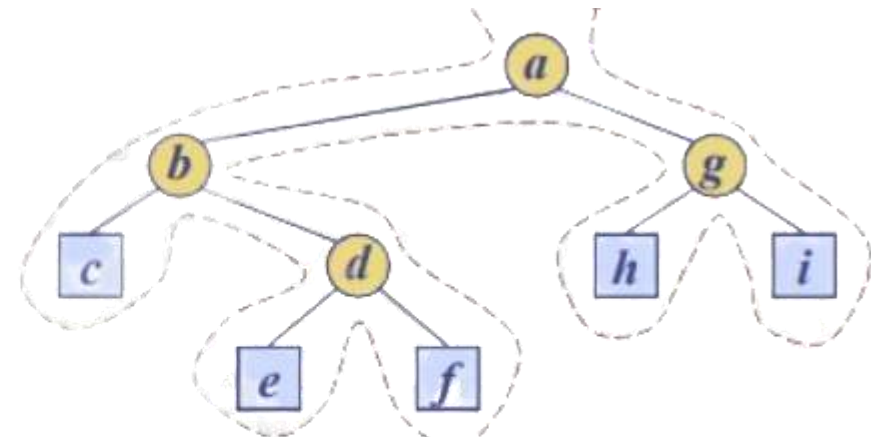
- Generic traversal of a binary tree
- Includes a special cases the preorder, postorder and inorder traversals
- Walk around the tree and visit each node three times:
 - on the left (preorder)
 - from below (inorder)
 - on the right (postorder)

Preorder: *a b c d e f g h i*

Inorder: *c b e d f a h g i*

Postorder: *c e f d b h i g a*

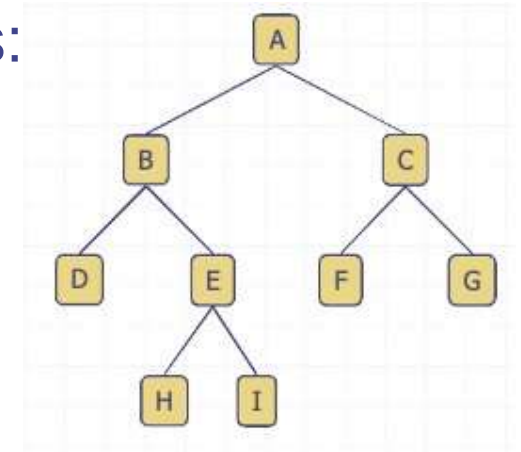
Euler Tour: *a b c b d e d f d b a g h g i g a*



Pre: *a b c d e f g h i*
In: *c b e d f a h g i*
Post: *c e f d b h i g a*

(Proper) Binary Tree

- A **(proper) binary tree** is a tree with the following properties:
 - Each internal node has two children
 - The children of a node are an ordered pair
- We call the children of an internal node left child and right child
- Alternative recursive definition:
 - A (proper) binary tree is either
 - a tree consisting of a single node, or
 - a tree whose root has an ordered pair of children, each of which is a proper binary tree

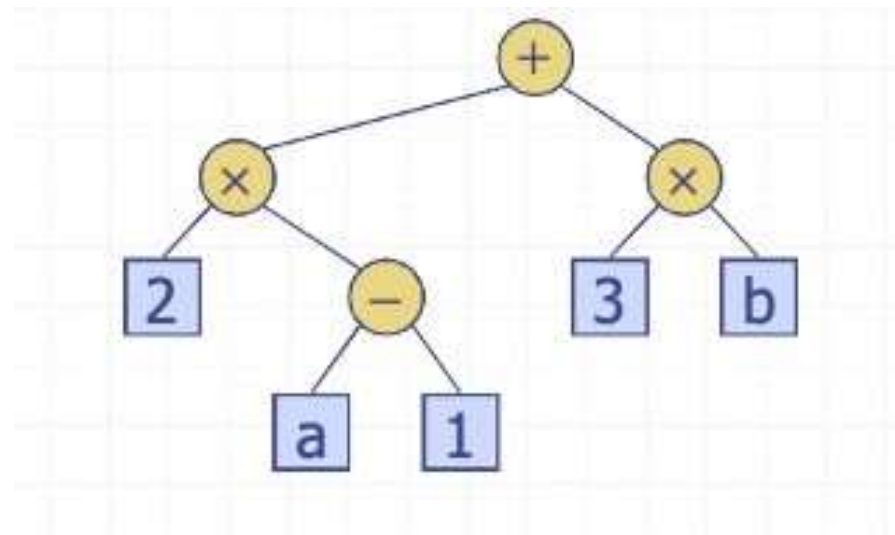


Application:

- arithmetic expressions
- decision processes
- searching

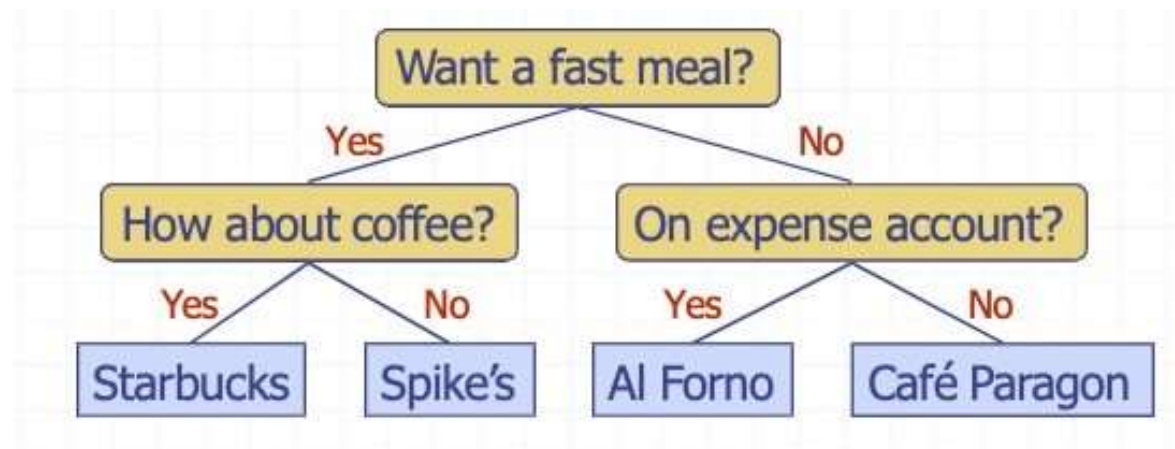
Arithmetic Expression Tree

- Binary tree associated with an arithmetic expression
 - internal nodes: operators
 - external nodes: operands
- Example: arithmetic expression tree for the expression $(2 * (a - 1) + (3 * b))$



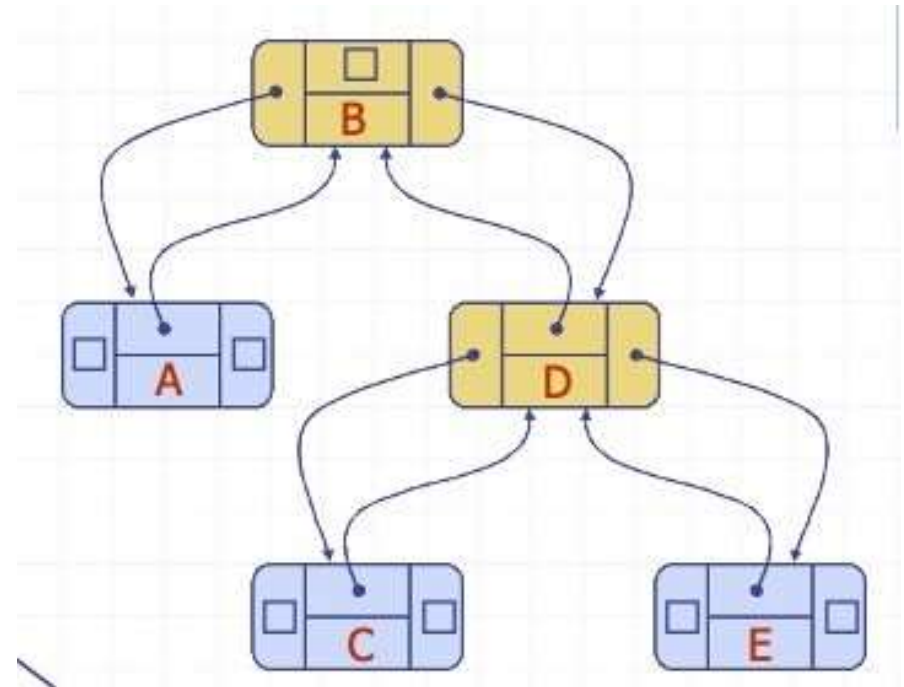
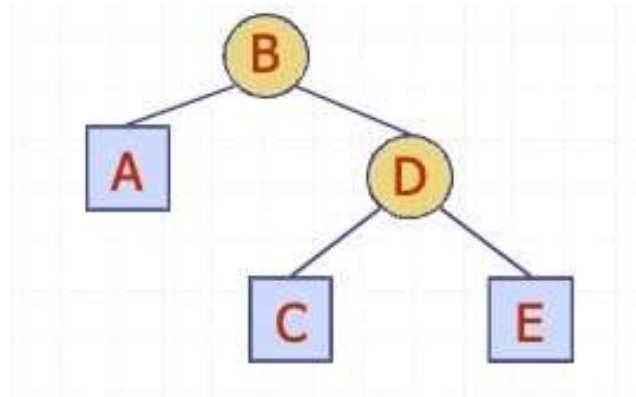
Decision Tree

- Binary tree associated with a decision process
 - internal nodes: questions with yes/no answer
 - external nodes: decisions
- Example: dining decision



Linked Structure for Binary Trees

- A node is represented by an object storing
 - Element
 - Parent node
 - Left child node
 - Right child node



Codes for Creation of Binary Trees

A binary tree can be created recursively.

The program will work as follow:

- Read a data in x .
- Allocate memory for a new node and store the address in pointer p .
- Store the *data* x in the *node* p .
- Recursively create the left subtree of p and make it the left child of p .
- Recursively create the right subtree of p and make it the right child of p .

Codes for Creation of Binary Trees

```
#include<stdio.h>
typedef struct TreeNode {
    struct TreeNode *left;
    int data;
    struct TreeNode *right;
} TreeNode;
```

Codes for Creation of Binary Trees

```
TreeNode * createBinaryTree( ){
    TreeNode *p;
    int x;
    printf("Enter data(-1 for no data): ");
    scanf("%d", &x);

    if(x == -1) return NULL;
    //create current node
    p = (TreeNode*) malloc(sizeof(TreeNode));
    p->data = x;
    //recursively create left and right subtree
    printf("Enter left child of %d: \n", x);
    p->left = createBinaryTree ( );
    printf("Enter right child of %d: \n",x);
    p->right = createBinaryTree ( );
    return p; }
```

Codes for Creation of Binary Tree Traversal

```
void preorder(TreeNode *t) {
    if(t != NULL) {
        printf("\n%d", t->data);
        preorder(t->left);
        preorder(t->right);
    }
}

int main() {
    TreeNode *root;

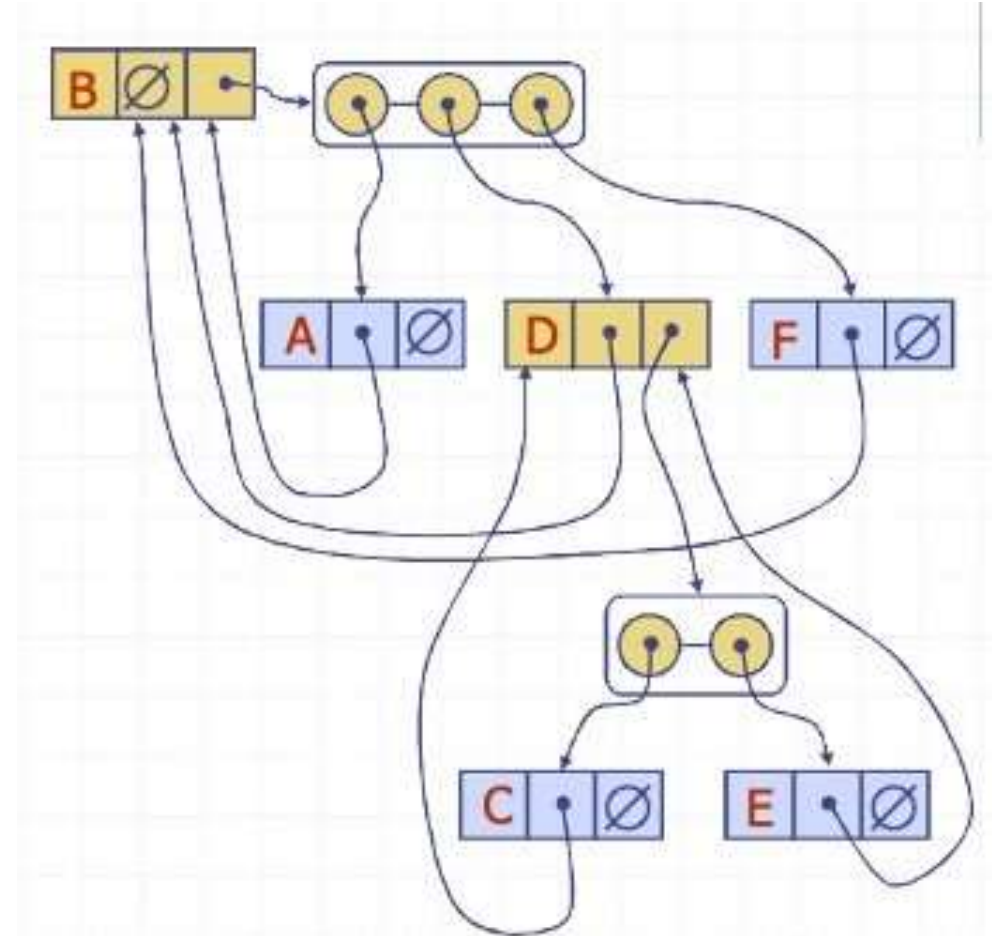
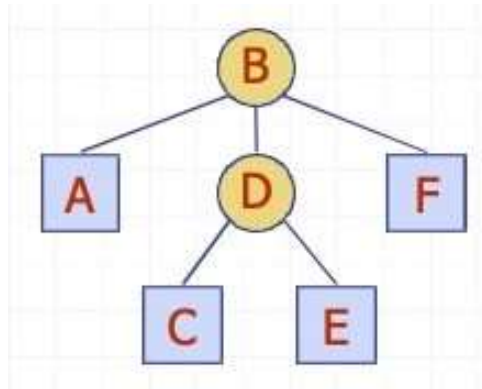
    root = createBinaryTree ( );

    printf("\nThe preorder traversal of tree is: \n");

    preorder(root);
}
```

LinkeSd Structure for General Trees

- A node is represented by an object storing
 - Element
 - Parent node
 - Children Container: Sequence of children nodes



Thank You